

Article

An Efficient Greedy Hierarchical Federated Learning Training Method Based on Trusted Execution Environments

Jiaxing Yan ^{1,2}, Yan Li ¹, Sifan Yin ¹, Xin Kang ¹, Jiachen Wang ¹, Hao Zhang ¹  and Bin Hu ^{1,*}

¹ School of Artificial Intelligence, Nanjing Agricultural University, Nanjing 210031, China; infojx@njau.edu.cn (J.Y.); 2024819030@stu.njau.edu.cn (Y.L.); 13603259281@163.com (S.Y.); 19221124@stu.njau.edu.cn (X.K.); 19421130@stu.njau.edu.cn (J.W.); 9213020417@stu.njau.edu.cn (H.Z.)

² Henan Key Laboratory of Network Cryptography Technology, Zhengzhou 450001, China

* Correspondence: hubin@njau.edu.cn

Abstract: With the continuous development of artificial intelligence, effectively solving the problem of data islands under the premise of protecting user data privacy has become a top priority. Federal learning is an effective solution to the two significant dilemmas of data islands and data privacy protection. However, there are still some security problems in federal learning. Therefore, this study simulates the data distribution in a hardware-based trusted execution environment in the real world through two processing methods: independent identically distributed and non-independent identically distributed methods. The basic model uses ResNet164 and innovatively introduces a greedy hierarchical training strategy to gradually train and aggregate complex models to ensure that the training of each layer is optimized under the premise of protecting privacy. The experimental results show that under the condition of an IID data distribution, the final accuracy of the greedy hierarchical model reaches 86.72%, which is close to the accuracy of the unpruned model at 89.60%. In contrast, under the non-IID condition, the model's performance decreases. Overall, the TEE-based hierarchical federated learning method shows reasonable practicability and effectiveness in a resource-constrained environment. Through this study, the advantages of the greedy hierarchical federated learning model with regard to enhancing data privacy protection, optimizing resource utilization, and improving model training efficiency are further verified, providing new ideas and methods for solving the data island and data privacy protection problems.

Keywords: federated learning; TEE; ResNet164; greedy layered training



Citation: Yan, J.; Li, Y.; Yin, S.; Kang, X.; Wang, J.; Zhang, H.; Hu, B. An Efficient Greedy Hierarchical Federated Learning Training Method Based on Trusted Execution Environments. *Electronics* **2024**, *13*, 3548. <https://doi.org/10.3390/electronics13173548>

Academic Editor: Daniele Riboni

Received: 6 August 2024

Revised: 31 August 2024

Accepted: 2 September 2024

Published: 6 September 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Data islands and data privacy [1] protection are two major dilemmas in artificial intelligence. Since artificial intelligence requires vast volumes of data, achieving rapid technological advancements by relying solely on a single institution's data is impractical. Therefore, establishing a connection between data, interconnecting data to form a joint force, and improving the utilization rate of the data are the aims of most current applications. However, the reality often differs from the ideal; adequate amounts of data are often challenging to obtain or are present as 'data islands'. Most companies' data sharing requires user consent, which many users refuse to provide, and the existence of internet giants has allowed a small number of companies to monopolize large amounts of data. Effectively solving the problem of data islands under the premise of protecting the data privacy of various companies and users and, on this basis, promoting the efficiency and accuracy of AI systems are top priorities. Therefore, in this context, federal learning came into being.

Federated learning (FL) [2] is an encrypted distributed machine learning model that has emerged recently. It allows multiple participants to jointly build and train machine learning models under the premise of protecting data privacy. The core advantage of this technology is that it can realize the standard training of the model through encrypted

parameter exchange without sharing the original data, thus solving the problem of data islands and meeting the requirements of data privacy protection and compliance. It was proposed by Google in 2016 and initially used to solve the problem of local update models for Android mobile phone end users. The design aims to enable efficient machine learning among multiple participants or computing nodes, ensuring data security and privacy and legal compliance. Federated learning allows participants to collaborate on AI projects without leaving local data. While protecting the privacy and security of all parties, the efficiency of the AI model is continuously improved. This solves the two significant dilemmas of data islands and privacy protection.

Different data dimensions divide federal learning into vertical federal learning [3,4], horizontal federal learning, and federal transfer learning [3,4]. In horizontal federated learning, where the user features of the two datasets overlap more and the user overlaps less, the dataset is segmented horizontally, and the portion of the data with the same user characteristics and different users is taken out for training. In vertical federated learning, with more overlapping users and fewer overlapping user features in the two datasets, the dataset is segmented vertically, and the portion of the data with the same users and different user features is taken out for training. Federated transfer learning does not segment the data when the user and user features of the two datasets are less overlapping and uses transfer learning to overcome the lack of data or labels.

However, the current federal learning model still has security problems. Federal learning needs more visibility for local training. It may be subject to attacks, such as data reconstruction attacks, attribute inference, or member inference attacks, which reduce the accuracy of the training model [5]. In the process of federated learning, when implementing its main tasks, the model will also learn information unrelated to its main tasks from user training data such that the attacker can detect the sensitive information of the parameter model itself and then launch an attack. In order to deal with this situation, the following methods were introduced. First, homomorphic encryption [6] was introduced, which is an encryption method that allows for some specific operations to be performed directly on encrypted data, and the result of the operation is consistent with the same operation on the original data after decryption. Data can be processed and analyzed without decryption, thereby protecting data privacy. However, it only supports limited arithmetic operations in the encrypted domain, which limits the application of homomorphic encryption in some complex computing scenarios. Second, multi-party computation [7] is a technology that allows multiple participants to complete a specific calculation together while protecting the privacy of their input. It allows data owners to jointly conduct data analysis and decision-making without leaking the original data. Other methods generate a lot of computational overhead. The need to ensure privacy through complex protocols often involves additional computational steps and communication costs, resulting in reduced efficiency. Third, differential privacy [8] is a technology that adds randomness to data analysis to protect personal privacy. By adding noise to the data, differential privacy can ensure that any individual's information cannot be identified in statistical analysis, thereby protecting their privacy. The effect of differential privacy depends on a parameter called the 'privacy budget', which determines the amount of noise added. Under specific settings, differential privacy cannot provide sufficient privacy protection.

In their latest study, Abbas Yazdinejad et al. [9] proposed a new learning model based on privacy protection. They used a Gaussian mixture model and the Mahalanobis distance to aggregate Byzantine tolerance. They used homomorphic encryption to ensure the confidentiality of data, which led to a superior performance in privacy protection. With a focus on improving security and addressing the privacy threats of ICPS, Danyal Namakshenas et al. [10] combined additive homomorphic encryption (AHE) for privacy protection with advanced feature selection methods and Shapley values (SVs) for enhancing interpretability. This dramatically reduced the computational overhead and ensured data security and privacy. Ali Dehghantanha et al. [11] also proposed an auditable privacy-preserving federated learning (AP2FL) model to solve the problem that the traditional FL

method cannot provide sufficient privacy protection and faces challenges in processing non-independent identically distributed (Non-IID) training data. The AP2FL model ensures the training and aggregation process of the client and server is secure, effectively reduces the burden on the client and server, and ensures the integrity, transparency, fairness, and robustness of the FL process.

Some things could be improved regarding the above solutions. Hardware-based trusted execution environments (TEEs) are a promising method to prevent the above attacks.

A TEE [12] is a secure computing environment that protects code and data from external attacks, including attacks from operating systems, hardware, and other applications. It achieves this goal by creating an isolated execution environment inside the processor. The working principle of a TEE is divided into four aspects. An independent execution environment is created inside the processor, isolated from other applications and operating systems. Then, the security of data and code is protected by hardware encryption technology. Data and code are encrypted before entering the TEE and decrypted when leaving the TEE. Digital signatures and hash algorithms ensure that the code and data are not tampered with during execution.

A TEE can create a secure area on the central processor to provide vital confidentiality and integrity guarantees for any data and code it stores or processes. Since only secure environment code is trusted, the TCB is minimized. A larger TEE increases the attack surface, so it should be kept small, limiting memory space.

Several TEE technologies are available on the market, including ARM's TrustZone, Intel SGX (version 2.5.101.3), and the open portable trusted execution environment OP-TEE. Among them, ARM's TrustZone has no limit on the size of the TEE, and the size of the HiKey 960 board TEE is only 16MiB. SGX (Software Guard Extensions) is a software protection solution provided by Intel. Providing a series of CPU instruction codes allows for the creation of a private memory area (enclave) with high access rights using user code, including O.S., VMM, BIOS, and SMM, which cannot access the enclave privately. The data in the enclave are only decrypted by the hardware on the CPU when the CPU is calculated. Therefore, data security in SGX technology is independent of the software operating system and hardware configuration. Data leakage can be prevented more effectively if the hardware driver, virtual machine, and operating system are attacked and destroyed. Intel SGX allows the TEE to create a fixed-size secure area memory of 128 MB (about 90 MB of applications are available). This induces significant paging overhead when memory over the PRM limit is required. In addition, although programs in the enclave cannot directly access operating system services such as system calls, system calls in the enclave will lead to enclave/non-enclave mode conversion, which will also lead to performance degradation, according to the latest research results. For the central server of federated learning, in the process of federated modeling, its ability to resist malicious node attacks is weak, and participants' contributions cannot be fully guaranteed to be positive. The latest issue in current research is identifying malicious nodes and reducing their impact. At the same time, the ideal state of federated learning is a completely decentralized joint modeling framework. However, complete decentralization still needs to be improved in current studies, and many business scenarios require a central server.

In order to make federated learning more efficient and able to effectively handle various technical problems under the memory constraints of TEEs, an efficient federated learning model based on a TEE is established in this study, and the optimization algorithm becomes the critical solution.

2. Materials and Methods

2.1. Purpose and Meaning

This study aims to comprehensively optimize the current resource-constrained federated learning challenges with a trusted execution environment (TEE) as the core support combined with an innovative hierarchical neural network training strategy. Specifically,

the objectives of this study include enhancing data privacy and security by leveraging the hardware-level isolation of a TEE, providing robust security against data leaks, reducing dependency on specific hardware, and improving the scheme's versatility and adaptability.

Secondly, optimization algorithms and resource utilization are optimized. Given the limited TEE memory, an innovative hierarchical training strategy is introduced to aggregate complex models layer-by-layer securely. This optimizes the TEE memory usage and reduces storage burdens. Thirdly, a federated learning platform based on the TEE is built; a federated learning platform based on Intel SGX is built to integrate the optimized hierarchical neural network training strategy and TEE security mechanism. The platform simplifies the implementation process of federated learning, providing a secure data transmission interface, efficient model aggregation, and flexible resource management to ensure both efficiency and security in model training.

Based on the existing hardware mechanism, the disadvantages linked to the TEE's strong dependence on the underlying hardware architecture are reduced. A 'greedy' hierarchical training method is adopted, dividing the ResNet164 model into three layers and placing them in the TEE progressively from shallow to deep for secure aggregation. This optimizes the TEE space usage and enhances the model security and efficiency, significantly reducing storage requirements, especially in resource-constrained federated learning scenarios.

2.2. Dataset

In this study, the CIFAR-10 dataset was used as the experimental basis of the federated learning algorithm. The dataset contains 60,000 32×32 color images covering ten categories. Each category has 6000 images, of which 50,000 are used for training and 10,000 for testing. It has proven to be an adequate benchmark dataset for federated learning research. In order to simulate the heterogeneity of data distributions in the real world, we performed independent identically distributed (IID) and non-independent identically distributed (Non-IID) processing on the CIFAR-10 dataset. IID and non-IID are concepts in probability theory and statistics usually used to describe a series of random variables.

Independent identically distributed (IID) processing: In order to ensure that the data sample categories received by each client are evenly distributed, that is, the dataset owned by each user is a subset of the entire dataset, and the category distribution between the subsets is similar, we randomly and non-repeatedly selected a specified number of samples for each user from all sample indexes to ensure the independence and uniformity of sample allocation.

Non-IID processing [13]: We first sorted the training set by label and divided the training set into 200 slices, each containing 250 samples. Each user randomly selected two shards, ensuring that the category distribution of data between users differed. This simulated the possible skewed distribution of data in the real world, that is, the uneven distribution of datasets for different clients.

2.3. Model

2.3.1. Basic Model

ResNet164 is a member of the deep residual network (ResNet) family and a variant of the deep learning model proposed by Kaiming He et al. in 2015 [14]. In the ImageNet and MS COCO competitions, ResNets with depths of more than 100 layers exhibited state-of-the-art accuracy in several challenging recognition tasks.

The original ResNet paper introduced multiple versions (ResNet18,34,50,101,152, etc.), but the commonly mentioned ResNet164 refers explicitly to a variant used in a specific study. It uses a bottleneck structure similar to that introduced after ResNet50; within each residual module, a smaller number of convolution kernels are used to reduce the computational complexity while maintaining the expression ability of the model.

ResNet164 has significant advantages over shallow models as a deep residual network family member.

ResNet164 solves the deep network degradation problem; with the increase in network depth, traditional neural networks often encounter the problem of performance saturation and even decline, that is, ‘deep network degradation’. ResNet enables the network to learn deeper representations without encountering serious degradation problems by introducing residual learning units. Each residual block allows the network to directly learn the residual between input and output. If the residual is zero, it means identity mapping, which ensures that the network can at least work like a shallow network, thus solving the problem that the deep network is challenging to train.

ResNet164 improves the model representation, and depth is one of the critical factors in improving the model’s ability to express. ResNet164 has a depth of 164 layers, which enables it to learn more complex feature representations. Compared with shallow models, it can capture multi-level abstract features in images or data to achieve better performance in image classification, target detection, and other tasks.

ResNet164 optimizes the training process; through the residual structure, ResNet164 can effectively alleviate the gradient disappearance and gradient explosion problems, making the model training more stable and faster. The residual connection is equivalent to providing a ‘highway’ for the gradient, ensuring that the gradient can be directly transmitted from the previous layer to the next layer and is not affected by the increase in network depth.

ResNet164 has a better generalization ability; the deep model typically performs better on unseen data due to its capacity to learn richer features, which means it performs better than shallow models on unseen data. Its computational efficiency is constantly improving. Although ResNet164 is deep, using techniques such as bottleneck design optimizes the use of computing resources while maintaining depth, ensuring the model is still competitive regarding computational efficiency.

Compared with shallow models, ResNet164 overcomes the challenges of deep network training through its unique residual structure and deep design. It significantly improves performance on complex tasks, becoming a milestone in deep learning.

Table 1 compares the ResNet164 model and other models regarding their effectiveness on the classification task.

Table 1. Comparison of ResNet model effects.

Dataset	Network	Baseline Unit	Pre-Activation Unit
CIFAR-10	ResNet-110 (1layer skip)	9.9	8.91
CIFAR-10	ResNet-110	6.61	6.37
CIFAR-10	ResNet-164	5.93	5.46
CIFAR-10	ResNet-1001	7.61	4.92
CIFAR-100	ResNet-1064	25.16	24.33
CIFAR-100	ResNet-1001	27.82	22.71

Figure 1 shows the ResNet164 model structure.

The two 1X1 convolutional layers in the bottleneck [15] are used to reduce and increase the feature dimension, respectively. The primary purpose is to reduce the number of parameters, thereby reducing the number of calculations required. After dimensionality reduction, data training and feature extraction can be performed more effectively and intuitively.

In deep learning, a ‘bottleneck’ refers to a network module or design that is mainly used to reduce the number of computations and parameters, thereby improving the performance and efficiency of the model. This design first appeared in ResNet and was widely used in ResNet v2. Conv means to carry out a convolution operation here. Among them, a convolution group from Conv to BatchNorm2d to ReLu in the table model includes one downsampling operation, which halves the size of the feature map and realizes the convolution operation through maximum pooling.

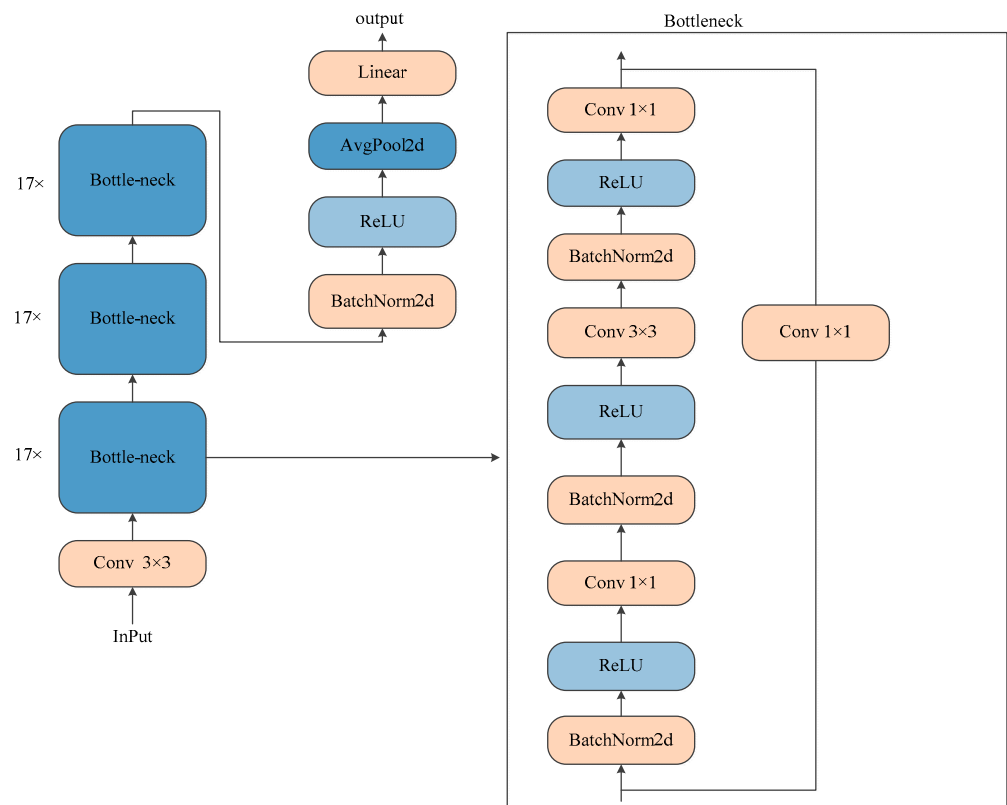


Figure 1. ResNet164 model structure diagram.

Specifically, the bottleneck design is used in ResNet to replace the traditional superficial convolution layer. The traditional convolutional layer applies larger filters (such as 3×3 or 5×5) at each position to obtain local features. However, such convolutional layers may sometimes generate too many calculations and parameters, especially in deep networks, leading to a slow training process, and they are prone to problems such as gradient disappearance or explosion.

The idea of the bottleneck design is to introduce a bottleneck layer, which consists of a series of filters of different sizes, usually 1×1 , 3×3 , and 1×1 convolutional layer sequences. This sequence first uses a convolution kernel of 1×1 to reduce the dimension, then uses a convolution kernel of 3×3 to extract the feature, and finally uses a convolution kernel of 1×1 to increase the dimension. This design can effectively reduce the dimensions of the feature map, thereby reducing the number of calculations and the number of parameters. In addition, the 1×1 convolutional layer can also be used to introduce nonlinear transformations. Such a structure enables the model to train and reason more efficiently while maintaining good performance, especially in deep networks.

2.3.2. Hierarchical Strategy

Previous studies have shown that greedy methods [16] can draw conclusions from analyzing shallow models, and greedy hierarchical methods can map these results to larger architectures. Compared with the traditional method, the greedy hierarchical strategy dramatically reduces the dependence on obtaining the entire gradient information. Most intermediate gradients do not need to be stored or computed, so they are instrumental in memory-constrained scenarios.

The core idea of the hierarchical greedy learning method is to decompose the training task of deep neural networks into multiple tasks involving the training of shallow networks. The entire network is built layer-by-layer, with each layer being an independently trained shallow module that relies on the previous layer's output as the input. By combining these modules, a deep network is ultimately formed.

Training starts with a shallow model until it converges. Then, a new layer is added to the converged model, and only this new layer is trained. Typically, a new auxiliary classifier is built for each added layer, which is used to output predictions and calculate the training loss. Therefore, these classifiers provide multiple exits for the inference process, with each layer corresponding to an exit.

In typical deep learning application scenarios such as image recognition [17], there are shared knowledge resources, such as pre-trained models or public datasets with similar characteristics to users' private data. These public resources are used as 'prior knowledge,' effectively guiding and accelerating the model training process. However, this knowledge is contained in the first layer of the model, which is usually responsible for capturing the basic features of the data, such as low-level visual elements such as edges and textures. These features are generally applicable to a variety of tasks. In particular, in deep models such as ResNet164, the initial layer has learned these essential and universal feature representations on large-scale datasets. These low-level features form the basis for more advanced abstractions in subsequent layers. Therefore, we freeze the pre-trained first-layer model parameters and only train the last few layers of the global model on the client side. This has several significant advantages: First, a reduced training burden. This avoids retraining these low layers on each client device, significantly reducing the consumption of computational resources, especially on resource-limited edge devices. Second, prevention of overfitting. Stable features trained on a wide range of data are retained, which helps reduce the risk of overfitting when the model faces private user data. Third, accelerated convergence. The model can quickly focus on high-level features related to specific tasks by fixing the known suitable feature extractor, accelerating the training process. Fourth, improve model consistency. It is ensured that all client models remain consistent regarding low-level feature extraction, which helps improve the overall coordination and model performance of federated learning.

In summary, the strategy of freezing the first-layer parameters of the model is based on the effective reuse of pre-training knowledge and acknowledging its utility. It aims to optimize resource utilization, accelerate training, and maintain the model's generalization ability. It is a strategy that can balance efficiency and privacy protection in federated learning.

Therefore, we designed a hierarchical strategy for the ResNet164 model: freezing the parameters of the first convolutional layer and dividing the three bottleneck modules into separate layers. The structure of the model after stratification is shown in Figure 2.

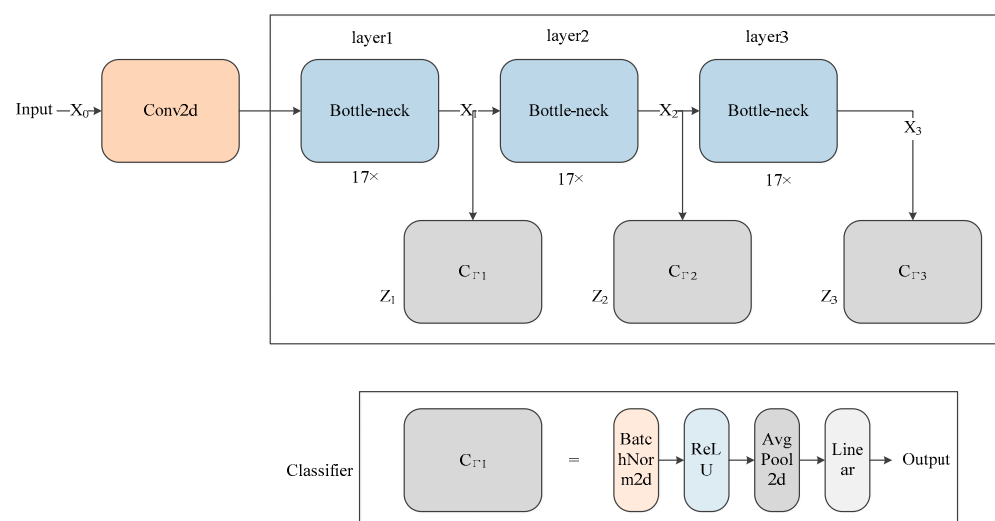


Figure 2. ResNet164 layered model structure diagram.

The tiering strategy is as follows: Firstly, the parameters of the first convolution layer are frozen (this layer does not participate in updates in all subsequent training steps; this is because the first layer is usually close to the data and can make better use of the low-level features of the pre-trained data). Secondly, the three bottleneck stages are divided into one layer each. Lastly, each layer is followed by an auxiliary classifier to output the prediction results for the current layer.

The training process is as follows: First, a network is built layer-by-layer. The initial input signal x_0 passes through the frozen convolution layer and enters the first layer of bottleneck operation, W_{θ_1} , to obtain the first layer output x_1 . The first layer output x_1 uses 329 as the input, and the second layer output x_2 is obtained by the second layer bottleneck operation W_{θ} . As mentioned above, the results of the upper layer are continuously recursively used as the input of the next layer until all layers of the model network are constructed. Secondly, for each layer j , the parameters of the previous layer, $\theta_1, \theta_2, \dots, \theta_{j-1}$, are fixed, and only the parameters of the current layer θ_j and the parameters of the auxiliary classifier γ_j are optimized. The output x_j of the current layer is classified by an auxiliary classifier C_{γ_j} , and the classification loss is calculated.

The optimization goal is to minimize the classification loss of the current layer:

$$\min_{\theta_j, \gamma_j} \frac{1}{N} \sum_{n=1}^N l(C_{\gamma_j}(x_j)y_n) \quad (1)$$

where l is the loss function (such as cross-entropy loss), x_j is the output of the current layer, and y_n is the corresponding label. The role of auxiliary classifiers is as follows: The output of the auxiliary classifier $z_{j+1} = C_{\gamma_j}(x_{j+1})$ is the prediction result of the current layer. By optimizing the loss of the auxiliary classifier, the feature extraction of each layer can be directly utilized to improve the expression ability of each layer. Among them, the Batchnorm and ReLU functions form a residual block group, and the output data are processed by the global average pooling layer (Avgpool) and output to the fully connected layer (Linear).

The hierarchical aggregation method is a commonly used clustering analysis method, through which clusters are formed by gradually merging or splitting data points. HAC is usually used in data mining and statistical analysis, especially when the precise number of clusters is not known. Its advantage is that it can avoid direct transmission and centralized data storage and protect data privacy. At the same time, the hierarchical aggregation method can also improve the accuracy and stability of the model because the model updates at different levels can complement each other to obtain a better global model.

In layer selection, the decision to freeze specific layers is based on their role in feature extraction. The initial layers, responsible for capturing fundamental features (e.g., edges and textures), are frozen. This prevents retraining these layers across different clients, which conserves computational resources and mitigates overfitting. The later layers, which capture more task-specific features, are unfrozen and optimized further. The optimization pathway is as follows: each layer is optimized sequentially by fixing the parameters of all previous layers and focusing the training on the current layer. This allows for a more manageable memory footprint, particularly in environments with limited resources like trusted execution environments (TEEs). The optimization objective at each step is to minimize the classification loss using an auxiliary classifier, ensuring that the features learned at each layer contribute effectively to the overall model performance.

It should be noted that in the hierarchical aggregation method, parameters, such as the number of layers and the importance of each layer, need to be adjusted according to the actual situation. In addition, in the hierarchical aggregation method, factors such as the computing power and communication bandwidth of the participants also need to be considered to maintain the training efficiency and accuracy of the model.

2.3.3. Model Application

We pre-trained and pruned the ResNet164 basic model and then designed its hierarchical model, which was finally applied in federated learning based on the Intel SGX trusted execution environment. Figure 3 shows the process of model application.

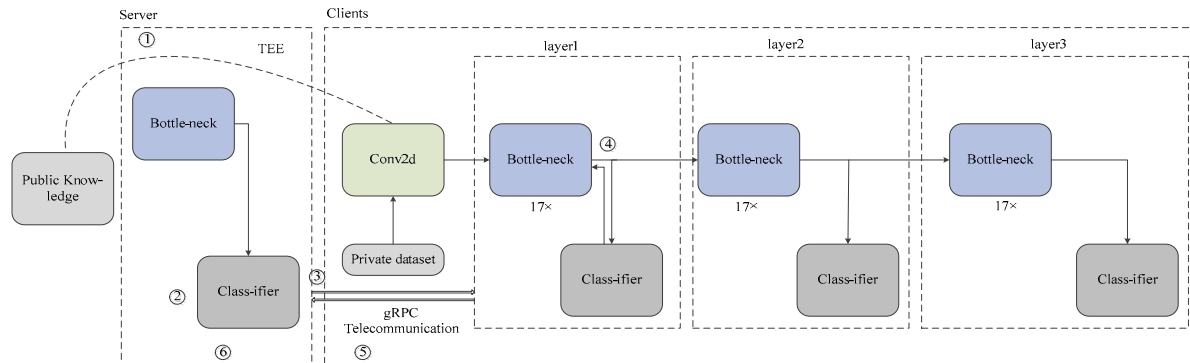


Figure 3. Schematic diagram of federal learning model based on a TEE.

①–② **Server public knowledge learning:** The original model uses the public dataset to learn, train, converge, and construct a pre-training model based on this dataset. Therefore, during initialization, the server selects a pre-trained model of public data which has a similar distribution to private data.

③ **Broadcasting specific layer parameters:** The server checks all available devices and constructs a set of participating clients to ensure that the TEE's memory is greater than the memory usage of these clients. Then, the layer parameters within the trained model are broadcast to these participating clients.

④ **After model transmission and configuration using gRPC remote communication,** each client model starts local training of its private data on this layer.

⑤ **After the client completes the local training of the layer,** all participating clients encrypt and upload the layer parameters to the server through GPRS remote communication.

⑥ **Finally,** the server safely aggregates and decrypts the received parameters in its TEE and applies the FedAvg algorithm to achieve aggregation, thereby safely generating a new global model layer.

The training of steps ③–⑥ of the global model is repeated until the training of all the layers of the hierarchical model is completed.

During the experiment, we observed the following characteristics of the hierarchical model: the parameters of the bottom layer proliferated, the correlation with the original features of the data weakened, and the data features were not vulnerable to attack.

Therefore, the following security decisions were made: the third-layer parameters were aggregated locally, TEE memory usage was optimized, overall security was ensured, and the computing efficiency and privacy protection were maintained.

2.4. System Environment

The development environment was based on an Ubuntu 20.04 LTS operating system with integrated Docker version 20.10.12 and Python 3.9.1. The processor was a 12th-generation Intel (R) Core (T.M.) i7-12700H system.

The trusted execution environment was Intel SGX SDK and Intel SGX PSW, processor technology launched by Intel. It extends the Intel architecture to provide enhanced code and data protection for security-sensitive applications on personal computers or servers. Its core concept is to create a protected memory area called an 'enclave,' which can ensure the confidentiality and integrity of code and data in the enclave even if the operating system, BIOS, driver, or higher-level software are maliciously attacked. However, enabling SGX requires hardware support and proper BIOS configuration. Since the launch of the sixth-generation Intel Core processor with the Skylake microarchitecture in 2015, SGX has

gradually become the focus of attention, although its popularity has been affected by factors such as BIOS support restrictions. Therefore, we installed the simulated versions of PSW and SDK for development.

2.5. SGX Framework

In this study, we used the Rust SGX framework, which is a Rust language development toolkit for Intel SGX's trusted computing platform. It allows programmers to use the Rust language to develop secure SGX-trusted programs quickly without memory security vulnerabilities. Even if the operating system is maliciously controlled, it can provide strong security protection capabilities to protect sensitive data from being stolen. This framework is of great significance for data privacy and cloud security. Its advantage is that it combines memory security, high performance, and a high degree of adaptation to security-critical areas. Rust's compile-time checking mechanism eradicates memory errors such as null pointer references and buffer overflows. This is critical for developing software in a secure execution environment such as SGX, ensuring it can resist attacks even in restricted environments. Rust's security concept coincides with SGX's original intention of ensuring data and code security. In addition, the Apache Teaclave SGX SDK, a toolkit designed specifically for SGX, helps us to build a safe and efficient SGX application, achieving a double improvement in security and development efficiency.

ECALLs (enclave calls) and OCALLs (out-of-enclave calls) are two core concepts related to Intel SGX. ECALLs are function calls initiated from an insecure area (regular application code) to a secure area (i.e., an enclave, an isolated memory area that guarantees the confidentiality and integrity of code and data). ECALLs allow application code requests to execute specific functions within a protected enclave. This process involves switching from a non-secure environment to a secure environment, ensuring that sensitive operations (data processing, key management, etc.) are performed in a protected environment, thereby preventing external malware or unauthorized access. OCALLs refer to function calls initiated inside the secure enclave to non-secure areas. OCALLs are used when code within the enclave needs access to external enclave resources or services (read files, network communications, system calls, etc.). Since the environment outside the enclave is not considered wholly trusted, the data transmitted through OCALL usually need to be encrypted, or other security measures are taken to ensure the security of the data after leaving the enclave. The enclave partition function call graph is shown in Figure 4.

Regarding memory management in the TEE, the following points are mainly used to solve memory constraints: Firstly, memory is allocated in advance. When creating an enclave, a certain amount of memory can be allocated to reduce the need for runtime memory allocation. This helps to reduce the performance overhead caused by memory allocation. Secondly, the memory page is managed. By using the page table to manage the memory page, the memory page can be loaded and released on demand. This on-demand paging mechanism can improve memory usage efficiency. Thirdly, memory is encrypted. Using memory encryption technology such as the AES-CTR mode can protect the enclave's memory data and prevent unauthorized access. At the same time, using an address as part of an encryption counter prevents data from being tampered with or replaced. Fourth, the memory integrity is protected. In addition to encryption, authentication mechanisms such as GMAC can be used to ensure the integrity of memory data and prevent data from being tampered with. Finally, memory access is controlled. Access to specific memory areas can be restricted using hardware-supported access control mechanisms, such as role-based access control (RBAC) or policy-based access control. Also, the use of memory is monitored in the enclave, the problem of memory usage over time is identified and solved, and the rationality and security of memory usage are ensured. Memory management using SGX has been performed here.

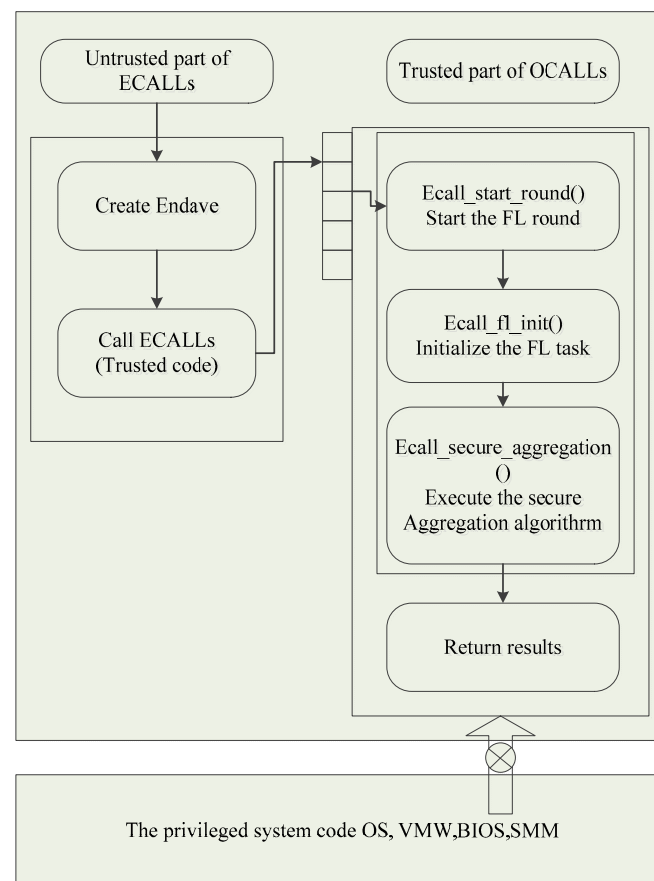


Figure 4. The enclave partition function call graph.

2.6. Communication Mechanism

gRPC communication is a kind of RPC remote procedure call. Calling remote functions is like calling local functions. It is necessary to define each API's request and response parameter formats.

gRPC has the following advantages: First, it delivers high performance. Using the HTTP/2 protocol and supporting features such as multiplexing and flow control, it is possible to efficiently transfer large amounts of data between the client and the server. At the same time, gRPC also uses platform-based optimization of serialization and deserialization techniques to improve communication efficiency. Secondly, it is easy to use. gRPC's IDL language is simple and easy to understand and provides a tool for automatically generating code, which is convenient for users to develop. The user only needs to define the IDL and generate the code to use a remote procedure call similar to a local function call in the code.

Moreover, gRPC has multilingual support. It supports various programming languages such as C++, Java, Python, Go, and Ruby, and RPC calls can be made between different programming languages. Regarding gRPC's scalability, gRPC supports a variety of extensions, including interceptors, load balancing, authentication, and authorization, which can meet the requirements of different scenarios. In terms of security, gRPC supports SSL/TLS secure transmission and provides a token-based authentication mechanism to ensure communication security. gRPC provides an efficient, scalable, multilingual, and secure RPC framework for inter-service communication in large-scale distributed systems.

Specifically, we use gRPC remote call to implement two functions: initiating aggregation requests (`call_grpc_aggregate`) and starting aggregation process requests (`call_grpc_start`):

The `call_grpc_aggregate` function is used to initiate aggregation requests to the server and receive multiple parameters, including the federated learning I.D. (`fl_id`), round, encrypted parameters, total number of parameters, the client I.D. list, the aggregation algorithm identifier, and the optimal number of clients. The function finally returns the

updated parameters, execution time, the I.D. list of clients participating in the aggregation, and the current round in the server response. The specific code is shown in Figure 5.

```
def call_grpc_aggregate(fl_id, round, encrypted_parameters, num_of_parameters,
    client_ids, aggregation_alg, optimal_num_of_clients):
    print(f'request 'aggregate' to {ADDRESS}')
    with grpc.insecure_channel(ADDRESS, options=options) as channel:
        stub = secure_aggregation_pb2_grpc.AggregatorStub(channel)
        response = stub.Aggregate(
            secure_aggregation_pb2.AggregateRequestParameters(
                fl_id=fl_id,
                round=round,
                encrypted_parameters=bytes(encrypted_parameters),
                num_of_parameters=num_of_parameters,
                aggregation_alg=aggregation_alg,
                optimal_num_of_clients=optimal_num_of_clients,
                client_ids=client_ids,
            )
        )
    return response.updated_parameters, float(response.execution_time), response.client_ids, response.round
```

Figure 5. Aggregation request function diagram.

The `call_grpc_start` function is used to start the secure aggregation process. The accepted parameters include the federated learning I.D., client I.D. list, noise standard deviation (σ), gradient clipping threshold (clipping), sampling ratio, aggregation algorithm identification, and number of parameters. The function finally returns the list of federated learning I.D.s, rounds, and participating client I.D.s in the server response, indicating the success of the startup request and its details. The specific code is shown in Figure 6.

```
def call_grpc_start(fl_id: int, client_ids: list, sigma: float, clipping: float,
    sampling_ratio: float, aggregation_alg: int, num_of_parameters: int):
    print(f'request 'start' to {ADDRESS}')
    with grpc.insecure_channel(ADDRESS) as channel:
        stub = secure_aggregation_pb2_grpc.AggregatorStub(channel)
        response = stub.Start(
            secure_aggregation_pb2.StartRequestParameters(
                fl_id=fl_id,
                client_ids=client_ids,
                sigma=sigma,
                clipping=clipping,
                sampling_ratio=sampling_ratio,
                aggregation_alg=aggregation_alg,
                num_of_parameters=num_of_parameters
            )
        )
    return response.fl_id, response.round, response.client_ids
```

Figure 6. Aggregate safety process function diagram.

2.7. Safety Assessment Analysis

In exploring federated learning systems based on trusted execution environments (TEEs), security analysis is critical in ensuring data privacy and integrity. Although a TEE provides an isolated execution environment for the secure processing of sensitive data, its security could be more robust and requires a comprehensive assessment. This emphasizes the need to focus on various attack vectors, including side-channel attacks (SCAs), and to maintain security in communications between clients and servers.

Side-channel attacks (SCAs) are fundamentally about acquiring ciphertext information through various leakage information generated during the operation of encryption software or hardware. For example, observing physical phenomena such as power consumption and electromagnetic radiation during system execution can infer sensitive information. The main advantage of a TEE is its hardware-based isolation mechanism, which restricts access to resources within the TEE, ensuring that even if the operating system or other software is compromised, the data or execution within the TEE cannot be spied on. Additionally, the code running within the TEE is subject to strict authentication, and only authorized code can be executed within the TEE, further enhancing its security.

To protect the security of communications between clients and servers, a TEE is often used in conjunction with encrypted communication protocols, such as gRPC (Google et al. Call), which is a high-performance, open-source, general-purpose RPC framework that supports multiple languages and allows for direct end-to-end encrypted communication between client and server applications. Using protocols like TLS and SSL, it is ensured that the data and model parameters generated within the TEE cannot be eavesdropped on or tampered with during transmission. A TEE also supports mutual authentication mechanisms to ensure that only legitimate clients and servers can participate in the federated learning process, preventing man-in-the-middle attacks.

2.8. System Flow

Intel's SGX technology plays a vital role in this study. It constructs a hardware-level secure enclave, namely, an enclave. In this way, even if there is a potential threat to the system software, data and algorithms can maintain their encryption state during the processing and only decrypt in a secure form within the CPU, which significantly alleviates the risk of data leakage and meets the high-standard requirements of federal learning for data privacy. The flow chart of federated learning based on a trusted execution environment is shown in Figure 7.

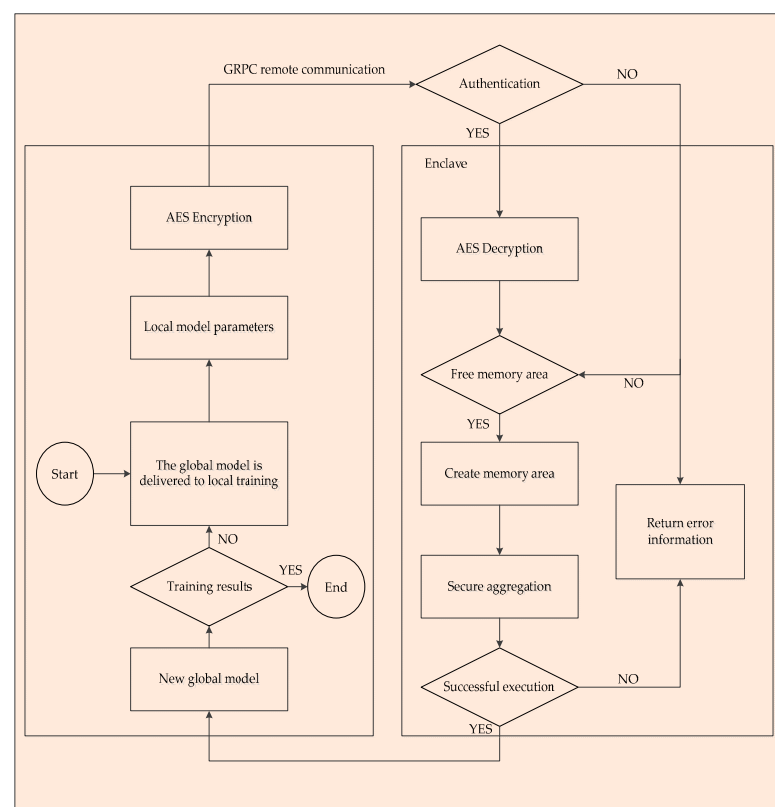


Figure 7. Federated learning flow chart based on a trusted execution environment.

The platform uses Intel SGX to ensure the security of the model aggregation process. The model's privacy is unaffected even if the server is not trusted. All data interaction processes are encrypted to ensure the confidentiality of communication.

3. Results

3.1. Pre-Training and Network Pruning

We first performed 160 rounds of local training on the ResNet164 model to save the model parameters. After pre-training, we used a network-slimming algorithm to prune it.

Network slimming is an advanced convolutional neural network (CNN) optimization method. Its core idea is to improve network performance by reducing the model's size and computing operations while maintaining or improving its accuracy. This method is especially suitable for those application scenarios with strict restrictions on model size and computing resources, such as mobile devices and embedded systems. The core of the network slimming method is to introduce channel-level sparsity in the training process. The channels of each convolutional layer are associated with a scaling factor from the Batch Normalization (B.N.) layer. These scaling factors play a role in measuring the importance of the channel. During the training process, by applying sparse regularization to these scaling factors, the network can automatically identify and mark the channels that contribute less to the model's performance, such as those with smaller scaling factor values. The pruning principle diagram of the model is shown in Figure 8:

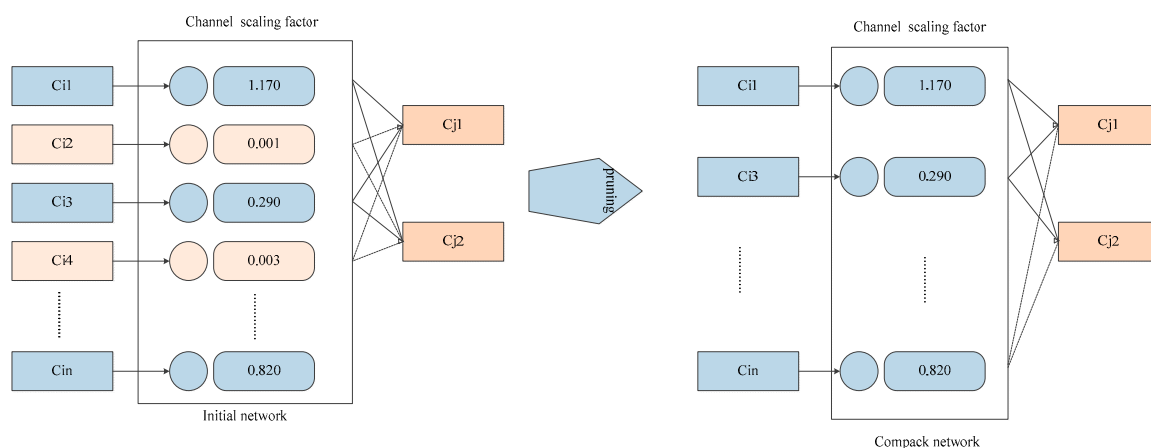


Figure 8. Pruning principle diagram of the model.

The steps are detailed as follows: First, the scaling factor is introduced. A scaling factor is introduced into each channel of the convolutional layer, which can reuse the scaling parameter (γ) of the Batch Normalization (B.N.) layer. In this way, the introduction of new parameters can be avoided, and the importance of the channel can be effectively measured. Secondly, sparse regularization is introduced in the training process. During the training process, sparse regularization is applied to these scaling factors. Specifically, by adding an L1 regularization term to the loss function to push the scaling factor to zero, the corresponding channel becomes unimportant. The training objective function is as follows:

$$L = \sum_{(x,y)} l(f(x, W), y) + \lambda \sum_{\gamma \in \Gamma} |\gamma| \quad (2)$$

where $l(f(x, W), y)$ is the normal training loss of the network, λ is a hyperparameter that balances training loss and sparse regularization, and Γ is a set of all scaling factors.

The specific code is shown in Figure 9.


```

def updateBN():
    for m in model.modules():
        if isinstance(m, nn.BatchNorm2d):
            m.weight.grad.data.add_(args.s*torch.sign(m.weight.data)) # L1

```

Figure 9. Regularization term function graph.

Finally, a pruning channel is introduced. After training, many scaling factors in the network will be close to zero. The channels with scaling factors close to zero can be clipped according to a global pruning threshold. The pruning threshold is usually set to a certain percentile of all scaling factors.

Once training is completed, the network slimming method will trim these less critical channels. This pruning process optimizes the network structure. The number of model parameters and its computational complexity can be significantly reduced by deleting channels that do not contribute much to the performance. Importantly, this pruning process does not significantly increase the overhead of the training process, and the pruned model can achieve memory and time savings without requiring special software or hardware accelerators.

After fine-tuning, the trimmed compact model can often achieve comparable or higher accuracy than the complete network. This is mainly due to the network slimming method effectively removing redundant and unnecessary parts while maintaining the model's performance.

The network slimming method can also be repeatedly applied to form a multi-network slimming strategy. The network can be compressed further to achieve a more efficient model by iterative pruning and fine-tuning. This strategy is especially suitable for application scenarios with strict requirements on model size, such as embedded devices or real-time systems.

In general, network slimming is an effective convolutional neural network optimization method, which reduces model size and computational operations by introducing channel-level sparsity while maintaining or improving the model's accuracy. This method is simple and easy and has unique advantages in various application scenarios. The key to this method is that the channel selection, guided by the scaling factor of the B.N. layer, can effectively reduce the demand for computing resources without sacrificing performance.

When setting the hyperparameters of our experiment, two are related to network slimming technology: the pruning rate (which is 0.7) and the sparse rate (which is 0.0001). As a result, we reduced the model parameters from 1.7 M to 1.3 M, cutting out 24% of the parameters, but the model's performance remained at a high level, similar to that before pruning. The changes in the number of channels before and after pruning are shown in Table 2.

Table 2. The channel number before and after pruning.

Model Layer Index	Number of Original Channels	Number of Reserved Channels
4	16	13
7	16	16
9	16	16
15	64	29
18	16	16
485	64	62
489	256	32
492	64	28
494	64	60
497	256	193

We performed local 10-round (batch size of 64) and global 15-round federal learning training on the pre-trained pruned and unpruned models to compare the performance of the pruned model. The results are shown in Table 3.

Table 3. Training index statistics under reduced parameter quantity.

Data Distribution	Pruning or Not	Number of Parameters	Training Loss	Training Accuracy	Test Loss	Test Accuracy
IID	YES	1.301 M	0.1613	96%	0.7841	91.17%
IID	NO	1.711 M	0.2539	100%	0.8902	89.60%
Non-IID	YES	1.301 M	0.2422	92%	1.5361	93.25%
Non-IID	NO	1.711 M	0.2429	98%	0.9126	89.09%

The experimental results showed that the model pruning successfully reduced the number of parameters under the IID data distribution and improved the model's performance. The training accuracy of the model after pruning reached 91.17%, which was 1.75% higher than that without pruning. This shows that the pruning did not reduce the model's generalization ability but improved its stability by reducing overfitting. In contrast, under the condition of the non-IID data distribution, the training and test performance of the model was reduced to varying degrees, and the pruning model was greatly affected by the data distribution.

3.2. Federated Learning Hierarchical Model Training

We divided the training dataset into 100 parts according to the settings in Section 1. We used one dataset for each client, and the experiment was conducted on two sets of data: (1) independent and identically distributed (IID), where the client has samples of all classes, and (2) non-independent identically distributed (Non-IID), where the client only has samples from two random classes.

Experiment 1. F.L. main hyperparameter settings: learning rate, 0.01; the proportion of users participating in aggregation in each round, 0.3; local training, 30 rounds; batch size, 64; and global training for 10 rounds. The results are shown in Tables 4 and 5.

Table 4. The training index statistics of IID test under the hierarchical model.

Layer	Number of Parameters	Training Loss	Training Accuracy	Test Loss	Test Accuracy	Aggregation Time
1	0.069 M	0.5126	62%	1.2205	63.20%	3.76 s(TEE)
2	0.268 M	0.396	78%	1.3293	63.87%	8.11 s(TEE)
3	0.978 M	0.1962	88%	0.7526	80.98%	GPU
Not layered	1.711 M	0.1178	100%	0.6173	93.53%	27.47(CPU)

Table 5. Training index statistics of Non-IID test under hierarchical model.

Layer	Number of Parameters	Training Loss	Training Accuracy	Test Loss	Test Accuracy	Aggregation Time
1	0.069 M	0.1704	16%	1.5083	46.52%	3.76 s(TEE)
2	0.268 M	0.1572	98%	1.5629	41.47%	8.11 s(TEE)
3	0.978 M	0.1633	64%	1.2229	66.32%	GPU
Not layered	1.711 M	0.1497	98%	0.8744	89.57%	27.47(CPU)

The final accuracy of the layered model was 80.98%, which was 13.42% lower than the original model's. From the test accuracy curve of each layer, it could be seen that there was an over-fitting phenomenon. Therefore, we reduced the number of local training arguments in the second experiment and increased the number of global training rounds.

In the case of non-independent and identically distributed data, the test accuracy of the last layer of the model reached 66.32%. The hierarchical model could have a better effect. However, the model still exhibited an upward trend in accuracy, so in the next experiment, we increased the number of global training rounds for each layer.

Experiment 2. F.L. main hyperparameter settings: learning rate, 0.01; the proportion of users participating in aggregation in each round, 0.3; local training, 10 rounds; batch size, 64; and global training for 15 rounds. The results are shown in Tables 6 and 7.

Table 6. Statistics of training indexes of IID test under hierarchical model after parameter change.

Layer	Number of Parameters	Training Loss	Training Accuracy	Test Loss	Test Accuracy	Aggregation Time
1	0.069 M	0.8726	68%	0.9752	68.13%	3.76 s(TEE)
2	0.268 M	0.8559	58%	0.8385	83.85%	8.11 s(TEE)
3	0.978 M	0.2689	96%	0.4237	86.72%	GPU
Not layered	1.711 M	0.2539	99%	0.8902	89.60%	27.47(CPU)

Table 7. Statistics of training indexes of Non-IID test under hierarchical model after parameter change.

Layer	Number of Parameters	Training Loss	Training Accuracy	Test Loss	Test Accuracy	Aggregation Time
1	0.069 M	0.2800	70%	1.3561	53.78%	3.76 s(TEE)
2	0.268 M	0.3435	58%	1.5170	48.59%	8.11 s(TEE)
3	0.978 M	0.3427	99%	1.5325	44.50%	GPU
Not layered	1.711 M	0.2519	94%	0.8659	89.66%	27.47(CPU)

The final accuracy of the hierarchical model was 86.72%, which was only 3.217% lower than that of the original model. However, the demand for storage space was significantly reduced during each aggregation, and the training time was also significantly reduced, which was crucial for the limited resources of the trusted execution environment. Therefore, despite the slight decrease in accuracy, the hierarchical model was still a better choice for TEE memory resource constraints.

However, in the case of non-independent identical distributions, the training accuracy of the last layer of the model was extremely high. However, the test accuracy was low, and each layer was lower than the previous layer. The layered model did not show a better effect. Compared with the non-layered model, the accuracy was reduced by 50.37%, and the accuracy curve fluctuated wildly. Therefore, the greedy hierarchical learning strategy may need to be improved to deal with uneven data distributions. We must optimize the algorithm in a complex data environment and find a breakthrough improvement method. We guess that part of the reason may be that under this Non-IID setting, because each client's dataset contains only a small number of samples of specific categories, it is difficult for the model to learn rich feature representations from global data during training. This may lead to a poor model performance in some categories, especially those that must be fully represented by the client. In addition, due to the different distribution of data for each client, the model may encounter difficulties in the aggregation phase, because updates from different clients may not be complementary but conflicting. For example, in the image classification task, client A may mainly contain pictures of a cat, while client B may mainly contain pictures of a dog. Such data distribution differences will lead to model training on each client focusing on different features.

4. Discussion

In terms of performance, in the case of similar data distributions, the accuracy of the greedy hierarchical model was 86.72%, which was close to the end-to-end federated learning effect and proves its effectiveness. In terms of the polymerization time, compared with CPU local aggregation, the greedy hierarchical aggregation strategy increased the aggregation

time by 56.79% and reduced the total duration of model parameter aggregation. In terms of communication and memory optimization, although the number of communication rounds was increased, the TEE memory allocation was optimized, and it was possible to safely aggregate important model parameters in batches, which strengthens its practicability in memory-constrained environments.

In the latest research, some scholars have proposed FedInverse, secure aggregation, SecureBoost security tree model, FATE, etc., to solve data privacy problems and data islands in federated learning. Secure aggregation [18] is a horizontal federated learning method based on secure aggregation. By adding noise before uploading model data and then controlling the noise distribution, the noises in the data will cancel each other after the aggregation of the model of multiple participants, thereby protecting privacy. FedInverse [19] is a method used to evaluate the risk of privacy leakages in federated learning. Using the Hilbert–Schmidt independence criterion (HSIC) as a regularizer to adjust the diversity of the model inversion (MI), the attack generator can effectively evaluate the risk of an attacker successfully obtaining data leakages from other participants. The SecureBoost security tree model [20] is a lossless, privacy-preserving lifting tree system based on federated learning. It allows multiple institutions to train the model jointly under data confidentiality and does not require the participation of an everyday trusted third party. These are the latest research results on the security of federated learning.

5. Conclusions

After several rounds of experimental evaluation, it was shown that the greedy hierarchical federated learning model had a final model accuracy of 86.72% when the data distributions were similar, which was only 3.217% lower than the accuracy of the original model. Thus, our model can approximately achieve the same effect as end-to-end federated learning. Although hierarchical federated learning increases the number of communication rounds required to complete all layers, it can improve the process of allocating memory in TEEs so that more large-scale model parameters can also be batched into TEE secure aggregation. Although the accuracy is slightly reduced, the hierarchical model is still a better choice for TEE memory resource constraints. However, we also realize that there is still room for optimization in the performance of existing models in complex scenarios with significant data bias, which signifies the direction for our follow-up research, i.e., the continuous exploration and improvement of algorithms in order to comprehensively improve the performance and robustness of the model for various data distributions while ensuring privacy. In summary, the effectiveness and practicability of greedy hierarchical federated learning are confirmed because the data distribution skew is small and the memory resources of the trusted execution environment are limited.

Author Contributions: J.Y. was mainly responsible for writing the thesis and preparing and collating the data. Y.L. mainly wrote the papers and made the model structure diagrams and carried out data collation with J.Y., B.H. and S.Y. The follow-up authors were mainly responsible for guiding techniques and writing methods. X.K., J.W. and H.Z. were responsible for the collection of the initial data and the corresponding training of the model. All authors have read and agreed to the published version of the manuscript.

Funding: This study was funded by the Basic Research Project of the Science and Technology Department of Qinghai Province, China (grant no. 2020-ZJ-716). This study was funded by the Henan Key Laboratory of Network Cryptography Technology (LNCT2022-A07). This study was funded by the Innovation Training Program for College Students of Artificial Intelligence College of Nanjing Agricultural University (202366XX017).

Data Availability Statement: The datasets used in the current study are available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Li, Q.; Wen, Z.; Wu, Z.; Hu, S.; Wang, N.; Li, Y.; Liu, X.; He, B. A survey on federated learning systems: Vision, hype, and reality for data privacy and protection. *IEEE Trans. Knowl. Data Eng.* **2021**, *35*, 3347–3366. [\[CrossRef\]](#)
- Mammen, P.M. Federated learning: Opportunities and challenges. *arXiv* **2021**, arXiv:2101.05428.
- Zhang, J.F.; Jiang, Y.C. A data augmentation method for vertical federated learning. *Wirel. Commun. Mob. Comput.* **2022**, *2022*, 6596925. [\[CrossRef\]](#)
- Zhuang, F.; Qi, Z.; Duan, K.; Xi, D.; Zhu, Y.; Zhu, H.; Xiong, H.; He, Q. A comprehensive survey on transfer learning. *Proc. IEEE* **2020**, *109*, 43–76. [\[CrossRef\]](#)
- Mothukuri, V.; Parizi, R.M.; Pouriyeh, S.; Huang, Y.; Dehghantanha, A.; Srivastava, G. A survey on security and privacy of federated learning. *Future Gener. Comput. Syst.* **2021**, *115*, 619–640. [\[CrossRef\]](#)
- Yi, X.; Paulet, R.; Bertino, E.; Yi, X.; Paulet, R.; Bertino, E. *Homomorphic Encryption*; Springer International Publishing: Berlin/Heidelberg, Germany, 2014.
- Goldreich, O. *Secure Multi-Party Computation*; Manuscript Preliminary Version; Weizmann Institute of Science: Rehovot, Israel, 1998; 108p.
- Dwork, C. Differential privacy. In *International Colloquium on Automata, Languages, and Programming*; Springer: Berlin/Heidelberg, Germany, 2006; pp. 1–12.
- Yazdinejad, A.; Dehghantanha, A.; Karimipour, H.; Srivastava, G.; Parizi, R.M. A robust privacy-preserving federated learning model against model poisoning attacks. *IEEE Trans. Inf. Forensics Secur.* **2024**, *19*, 6693–6708. [\[CrossRef\]](#)
- Namakshenas, D.; Yazdinejad, A.; Dehghantanha, A.; Parizi, R.M.; Srivastava, G. IP2FL: Interpretation-Based Privacy-Preserving Federated Learning for Industrial Cyber-Physical Systems. *IEEE Trans. Ind. Cyber-Phys. Syst.* **2024**, *2*, 321–330. [\[CrossRef\]](#)
- Yazdinejad, A.; Dehghantanha, A.; Srivastava, G. AP2FL: Auditable privacy-preserving federated learning framework for electronics in healthcare. *IEEE Trans. Consum. Electron.* **2023**, *70*, 2527–2535. [\[CrossRef\]](#)
- Schneider, M.; Masti, R.J.; Shinde, S.; Capkun, S.; Perez, R. Sok: Hardware-supported trusted execution environments. *arXiv* **2022**, arXiv:2205.12742.
- Tillman, R.E. Structure learning with independent non-identically distributed data. In Proceedings of the 26th Annual International Conference on Machine Learning, Montreal, QB, Canada, 14–18 June 2009; pp. 1041–1048.
- He, K.; Zhang, X.; Ren, S.; Sun, J. Identity mappings in deep residual networks. In *Computer Vision—ECCV 2016, Proceedings of the 14th European Conference, Amsterdam, The Netherlands, 11–14 October 2016*; Springer International Publishing: Berlin/Heidelberg, Germany, 2016; pp. 630–645.
- Koh, P.W.; Nguyen, T.; Tang, Y.S.; Musmann, S.; Pierson, E.; Kim, B.; Liang, P. Concept bottleneck models. In Proceedings of the International Conference on Machine Learning, PMLR, Online, 3–18 July 2020; pp. 5338–5348.
- Belilovsky, E.; Eickenberg, M.; Oyallon, E. Greedy layerwise learning can scale to imagenet. In Proceedings of the International Conference on Machine Learning, PMLR, Long Beach, CA, USA, 9–15 June 2019; pp. 583–593.
- He, K.; Zhang, X.; Ren, S.; Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778.
- Fereidooni, H.; Marchal, S.; Miettinen, M.; Mirhoseini, A.; Möllering, H.; Nguyen, T.D.; Rieger, P.; Sadeghi, A.R.; Schneider, T.; Yalame, H.; et al. SAFElearn: Secure aggregation for private federated learning. In Proceedings of the 2021 IEEE Security and Privacy Workshops (SPW), San Francisco, CA, USA, 27 May 2021.
- Wu, D.; Bai, J.; Song, Y.; Chen, J.; Zhou, W.; Xiang, Y.; Sajjanhar, A. FedInverse: Evaluating privacy leakage in federated learning. In Proceedings of the Twelfth International Conference on Learning Representations, Vienna, Austria, 7–11 May 2024.
- Cheng, K.; Fan, T.; Jin, Y.; Liu, Y.; Chen, T.; Papadopoulos, D.; Yang, Q. Secureboost: A lossless federated learning framework. *IEEE Intell. Syst.* **2021**, *36*, 87–98. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.